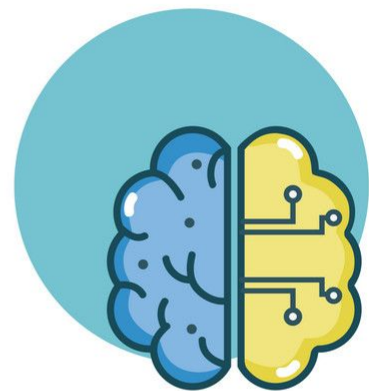


INTRODUCTION TO MACHINE LEARNING

INTRODUCTION TO NEURAL NETWORKS

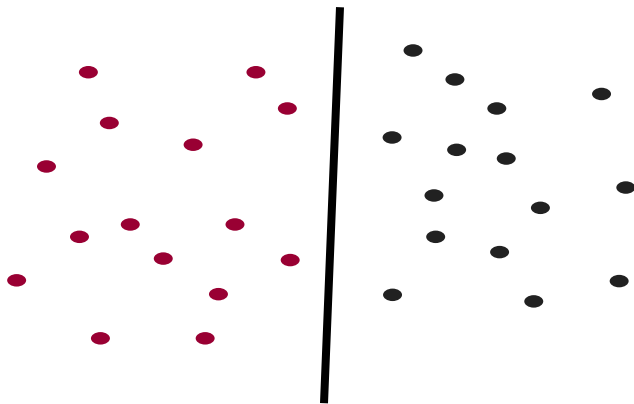


Elisa Ricci



LINEAR MODELS

- A **linear model** is a model that assumes the data is linearly separable
- We studied the **perceptron**



PERCEPTRON LEARNING ALGORITHM

repeat until convergence (or for some # of iterations):

for each training example $(x_{i1}, x_{i2}, \dots, x_{in}, y)$:

label is -1/1

$$\text{prediction} = \sum_j w_j x_{ij} + b$$

if *prediction is different from label*

for each w_j :

$$w_j = w_j + x_{ij} y_i$$

$$b = b + y_i$$

PERCEPTRON LEARNING ALGORITHM

repeat until convergence (or for some # of iterations):

for each training example $(x_{i1}, x_{i2}, \dots, x_{in}, y)$:

label is -1/1

$$prediction = \sum_j w_j x_{ij} + b$$

if $prediction \cdot y_i \leq 0$: // they don't agree

for each w_j :

$$w_j = w_j + x_{ij} y_i$$

$$b = b + y_i$$

PERCEPTRON LEARNING ALGORITHM

Algorithm 5 PERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $w_d \leftarrow 0$ , for all  $d = 1 \dots D$  // initialize weights
2:  $b \leftarrow 0$  // initialize bias
3: for  $iter = 1 \dots MaxIter$  do
4:   for all  $(x, y) \in \mathbf{D}$  do
5:      $a \leftarrow \sum_{d=1}^D w_d x_d + b$  // compute activation for this example
6:     if  $ya \leq 0$  then
7:        $w_d \leftarrow w_d + yx_d$ , for all  $d = 1 \dots D$  // update weights
8:        $b \leftarrow b + y$  // update bias
9:     end if
10:   end for
11: end for
12: return  $w_0, w_1, \dots, w_D, b$ 
```

Algorithm 6 PERCEPTRONTEST($w_0, w_1, \dots, w_D, b, \hat{x}$)

```
1:  $a \leftarrow \sum_{d=1}^D w_d \hat{x}_d + b$  // compute activation for the test example
2: return SIGN( $a$ )
```

PERCEPTRON LEARNING & GRADIENT DESCENT

repeat until convergence (or for some # of iterations):

for each training example $(x_{i1}, x_{i2}, \dots, x_{in}, y)$:

$$\text{prediction} = \sum_j w_j x_{ij} + b$$

if $\text{prediction } y_i \leq 0$: // they don't agree

**Note: for gradient descent,
we always update**

for each w_j :

$$w_j = w_j + x_{ij} y_i$$

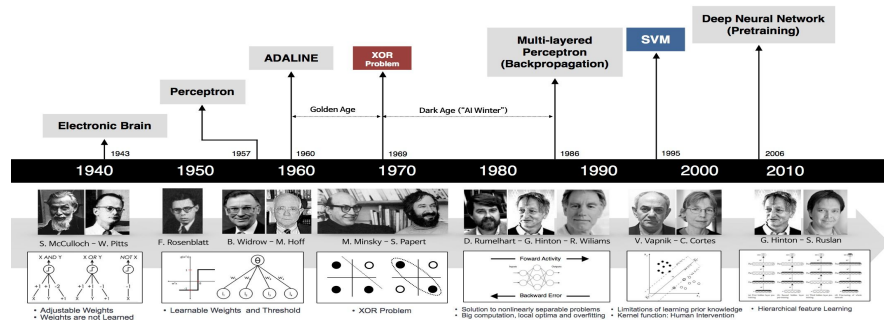
$$b = b + y_i$$

Perceptron
update

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$$

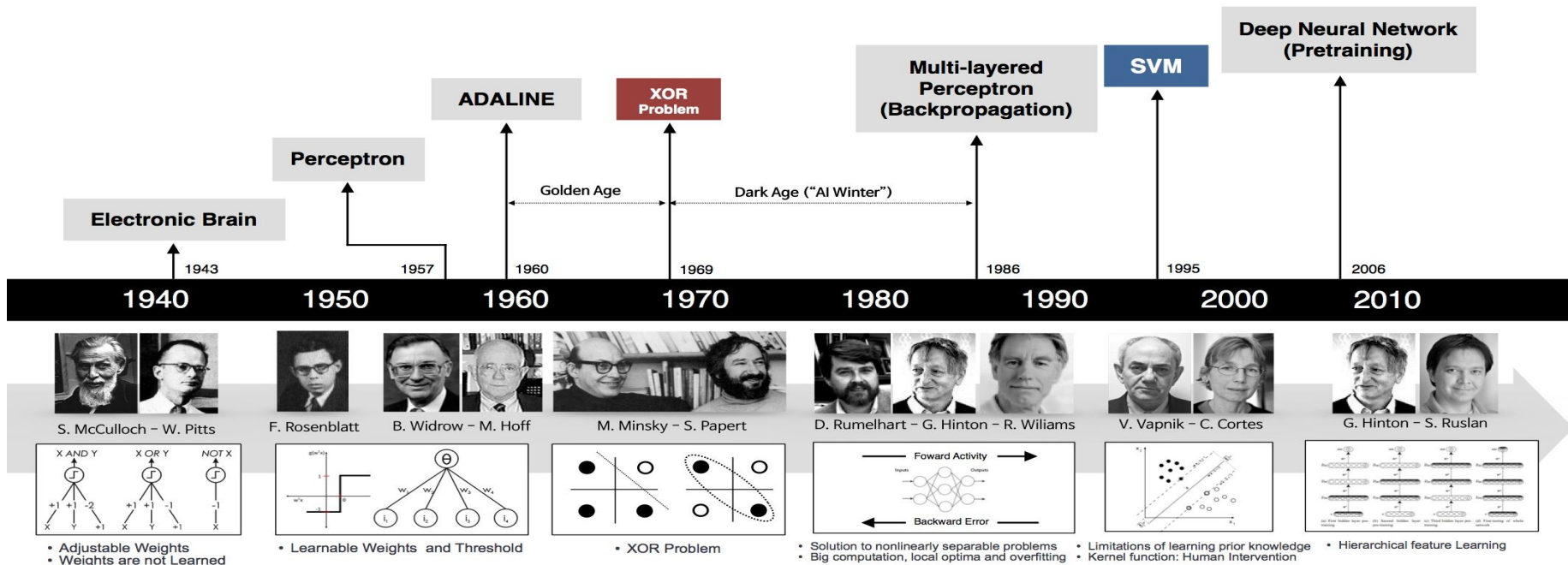
Gradient
descent update

In practice $w_j = w_j + x_{ij} y_i c$ where $c = \eta \exp(-y_i(w \cdot x_i + b))$



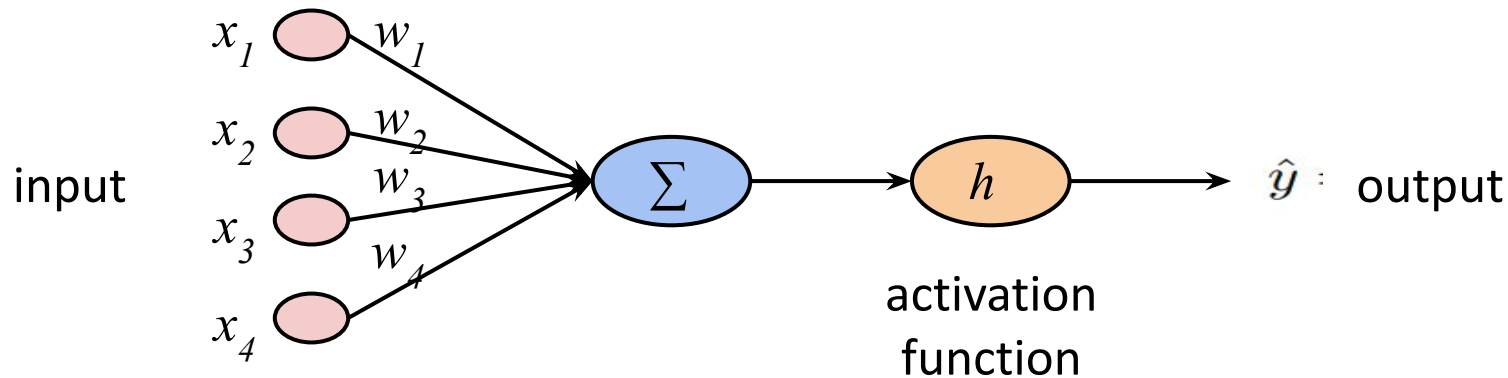
FROM PERCEPTRON TO DEEP NETWORKS

HISTORY OF NEURAL NETWORKS



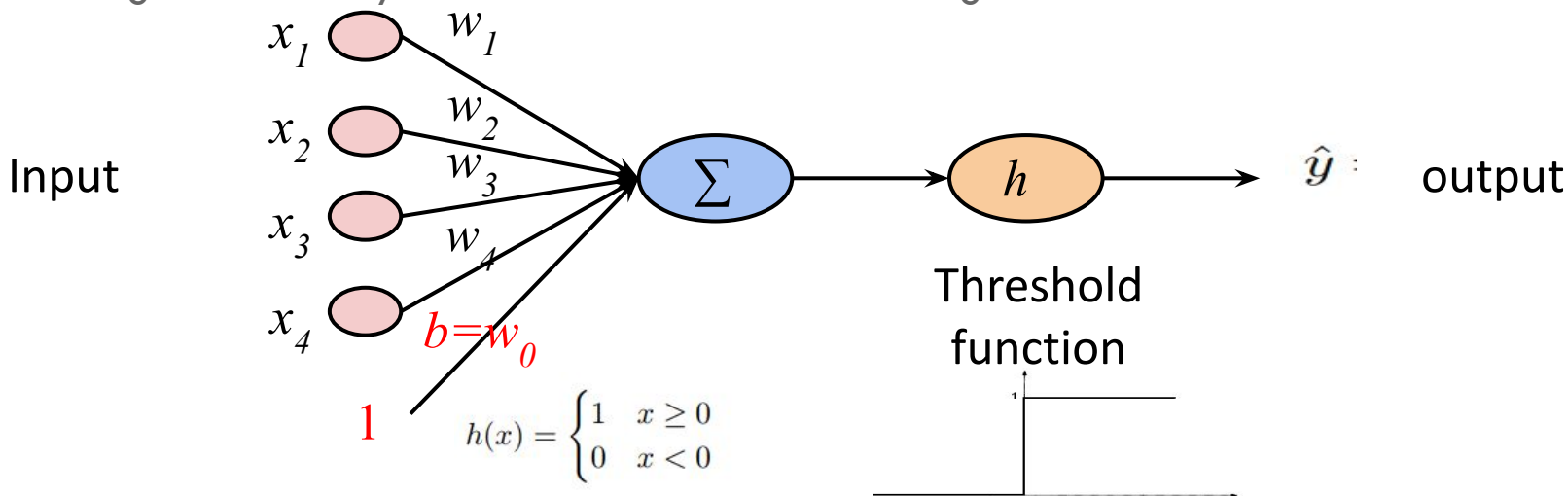
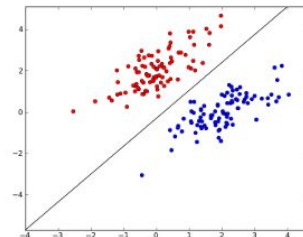
PERCEPTRON

An artificial neuron, i.e. a perceptron, is a non-linear parameterized function with restricted output range



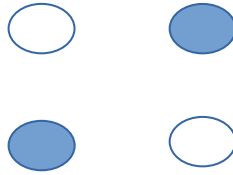
ROSEMBLATT (1958)

- Perceptron as a machine for linear classification
- Main idea: Learn the weights and consider bias.
 - One weight per input
 - Multiply weights with respective inputs and add bias.
 - If result larger than threshold return 1, otherwise 0
- The algorithm is very similar to what we saw with sign function

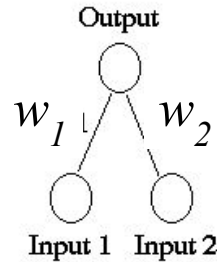


MINSKY & PAPERT (1969)

- The Exclusive OR (XOR) cannot be solved with a perceptron.

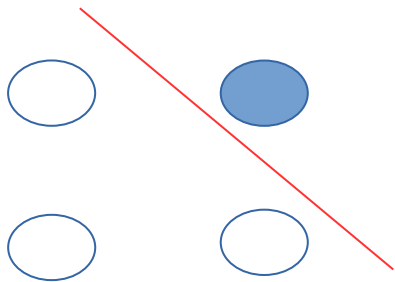


Input 1	Input 2	Output
1	1	0
1	0	1
0	1	1
0	0	0

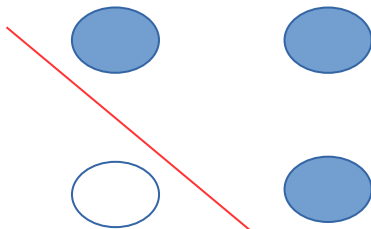


MINSKY & PAPERT (1969)

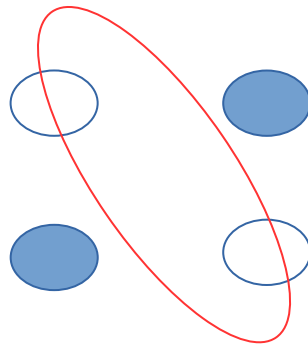
- Linearly separable and not separable problems



AND



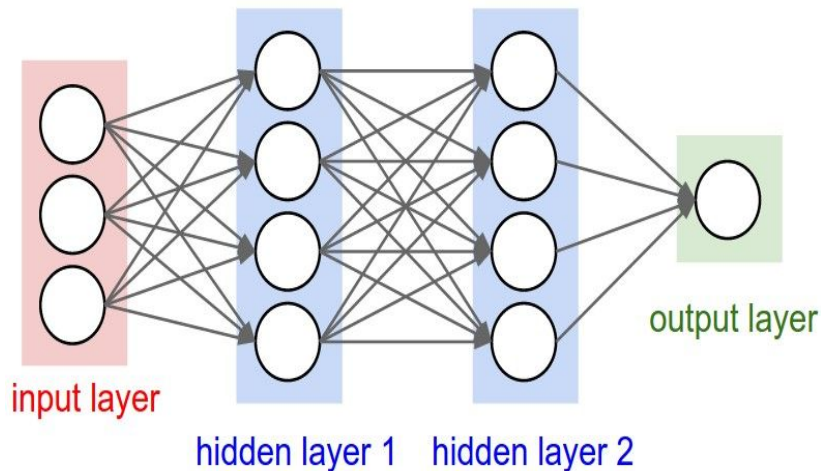
OR



XOR

MULTI-LAYER PERCEPTRON

- We can solve XOR with a Multi-Layer Perceptron (MLP)
- Idea: Densely connect artificial neurons to realize compositions of non-linear functions
- Can be used to arbitrary supervised learning tasks: classification, regression, etc.

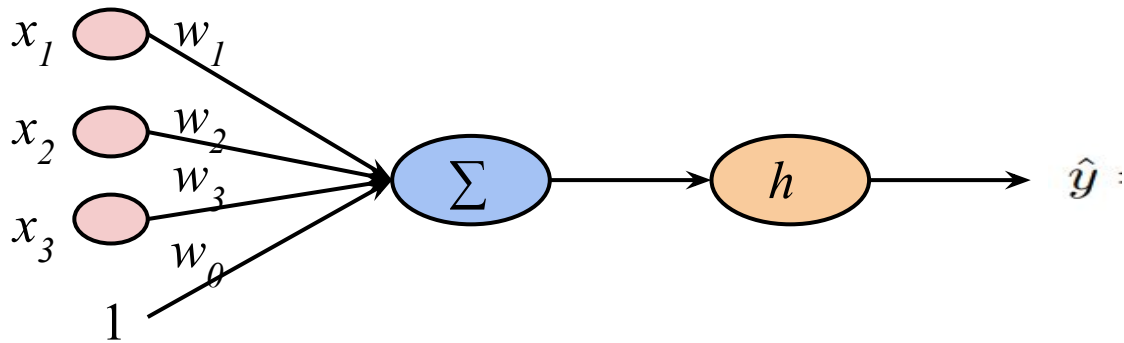


FEED-FORWARD NEURAL NETWORKS

- The information is propagated from the inputs to the outputs
- **Directed Acyclic Graph (DAG)**: no cycles between outputs and inputs
- Compute **one or more non-linear functions**
 - Computation is carried out by composition of some number of algebraic functions implemented by the connections, weights and biases of the hidden and output layers
- Hidden layers compute intermediate representations

FEED-FORWARD NEURAL NETWORKS

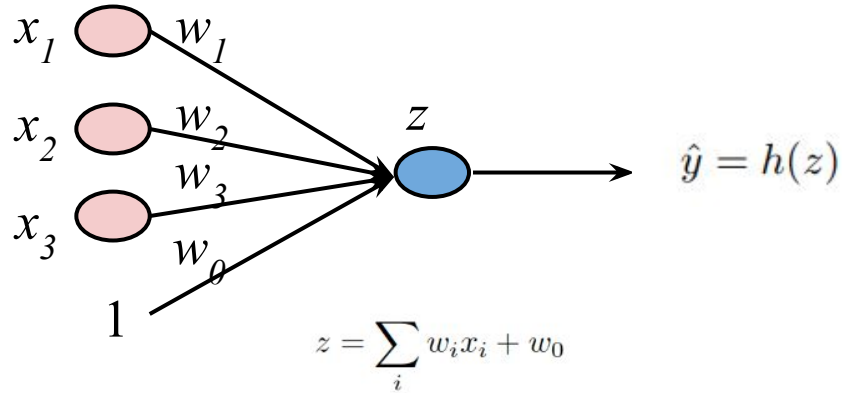
Perceptron



$$\hat{y} = h\left(\sum_i w_i x_i + w_0\right) = h(\mathbf{w}^T \mathbf{x} + w_0)$$

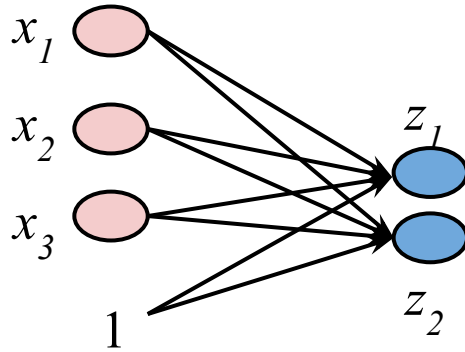
FEED-FORWARD NEURAL NETWORKS

Simplified model



FEED-FORWARD NEURAL NETWORKS

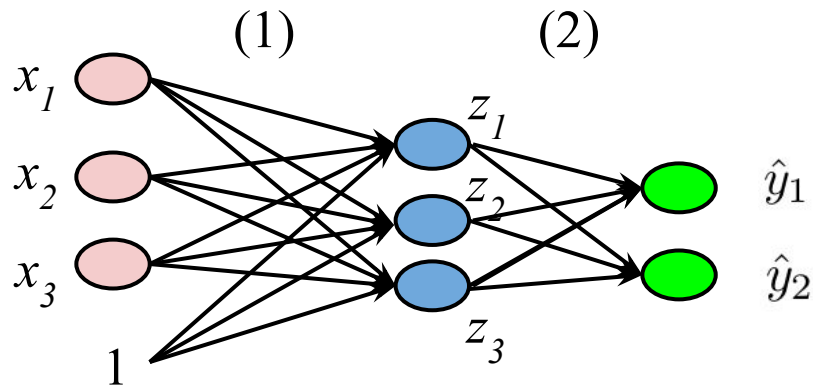
Multiple outputs



$$z_j = \sum_i w_{i,j} x_i + w_{0,j}$$

FEED-FORWARD NEURAL NETWORKS

Single layer neural network



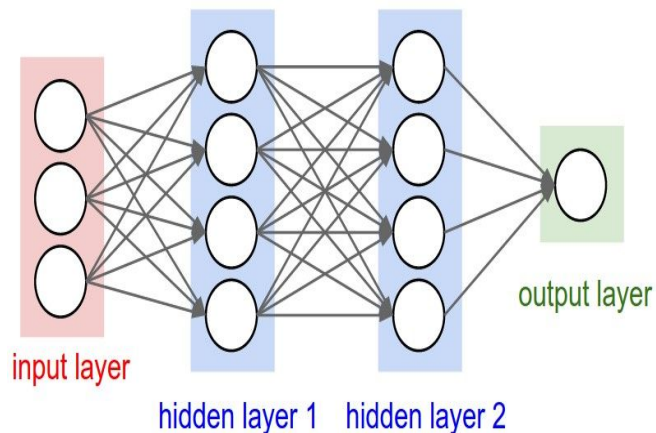
$$z_j = \sum_i w_{i,j}^{(1)} x_i + w_{0,j}^{(1)}$$

$$\hat{y}_k = f\left(\sum_i w_{i,k}^{(2)} h(z_i) + w_{0,k}^{(2)}\right)$$

FIRST AI WINTER

How to train a MLP?

- Perceptron learning rule is not applicable, as it expects to know the desired target
- For **hidden layers** we cannot know the desired target, for **output layers** we have direct supervision (from ground truth labels)

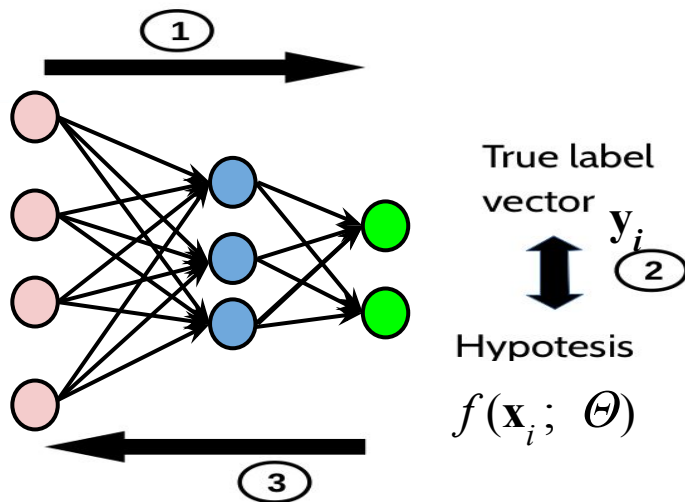


BACKPROPAGATION (1986)

- Backpropagation revitalized the field
- Learning MLP for complicated functions can be solved
- Efficient algorithm which processes “large” training sets
- Today backpropagation is still at the core of neural network training

BACKPROPAGATION IN A NUTSHELL

1. **Forward propagation:** sum inputs, produce activations, feed-forward
2. **Error estimation**
3. **Back propagate** the error signal and used it to update weights



BACKPROPAGATION IN A NUTSHELL

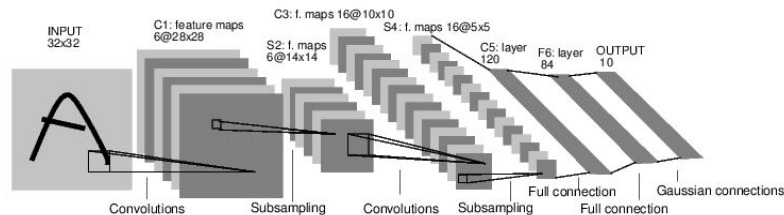
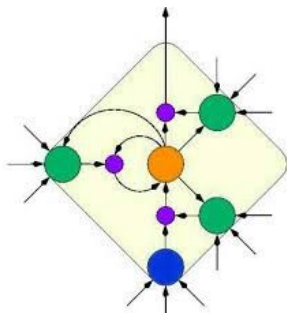
- **Learning = Optimization**
- Main idea: Given training samples $T = \{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots, (\mathbf{x}_N, \mathbf{y}_N)\}$ adjust all the weights of the network Θ such that a cost function is minimized

$$\min_{\Theta} \sum_i L(\mathbf{y}_i, f(\mathbf{x}_i; \Theta))$$

- Choose your loss function (e.g. square loss or any of the losses we have seen)
- Update the weights of each layer with **gradient descent**
- Use the BackPropagation of the error signal to **compute the gradient efficiently**

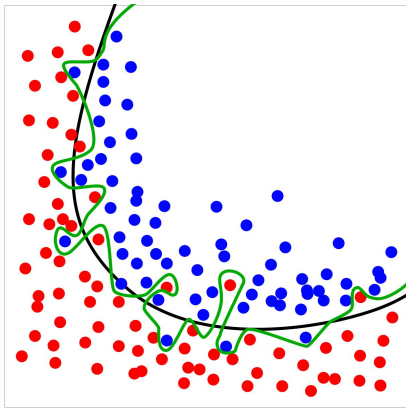
CNN & LSTM (1990)

- Important advances in the field:
 - Backpropagation
 - Recurrent Long-Short Term Memory Networks (Schmidhuber, 1997)
 - Convolutional Neural Networks: OCR solved before 2000s (LeNet, 1998).



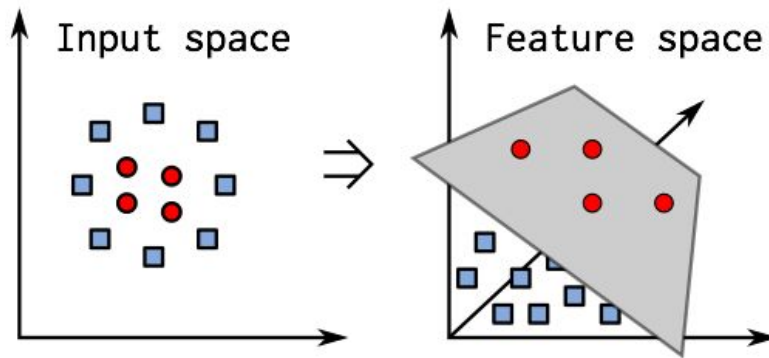
SECOND AI WINTER

- NN cannot exploit many layers
 - Overfitting
 - Vanishing gradient (with NN training you need to multiply several small numbers → they become smaller and smaller)
- Lack of processing power (no GPUs)
- Lack of data (no large annotated datasets)



SVM & KERNEL METHODS

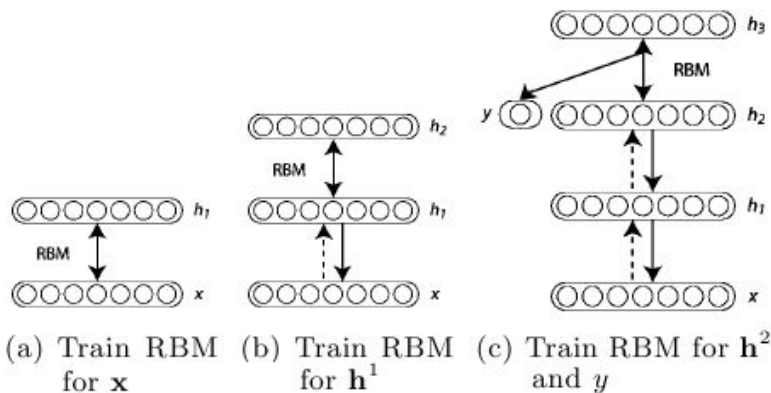
- Kernel Machines (e.g. SVMs) suddenly become very popular
 - Similar accuracies than neural networks in the same tasks
 - Much fewer heuristics and parameters
 - Nice proofs on generalization



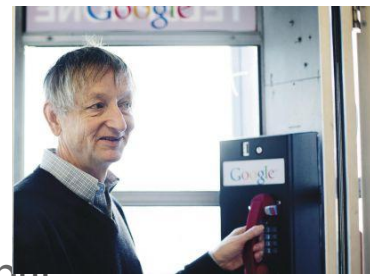
LEARNING DEEP BELIEF NETS (2006)



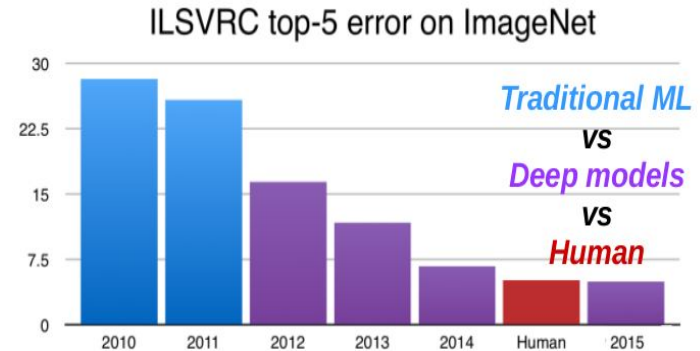
- Clever way of initializing network weights:
 - Train each layer one by one with unsupervised training (using contrastive divergence)
 - Much better than random values
- Fine-tune weights with a round of supervised learning
- State of the art performance on MNIST dataset



ALEXNET



- Hinton's group implemented a CNN similar to LeNet [LeCun1998] but...
 - Trained on ImageNet with two GPUs
 - With some technical improvements (ReLU, dropout, data augmentation)



DEEP LEARNING REVOLUTION

Illustration by Justin Metz
SEPTEMBER 28, 2016, 5:00 PM EDT

WHY DEEP LEARNING IS
SUDDENLY CHANGING YOUR LIFE

EE|Times

Connecting the Global
Electronics Community

Home News Opinion Messages Authors Video Slideshows Teardown Education EELife Events

About Us | Newsletter Sign Up

designlines Android Automotive Embedded Industrial Control Internet of Things MCU Medical Memory Open Source PCB

BREAKING NEWS

NEWS & ANALYSIS: Qualcomm Tips 28 GHz 5G Chip

INTRODUCING
POWER
ELECTRONICS NEWS
Power
Electronics
News.com

TECHONLINE
Education for
the creators of
technology



News & Analysis

Microsoft, Google Beat Humans at Image Recognition

Deep learning algorithms compete at ImageNet
challenge

R. Colin Johnson
2/16/2015 08:15 AM EST
14 comments

NO RATINGS
1 saves
LOGIN TO RATE

f Like 91 t Tweet in Share 48 G+ 44

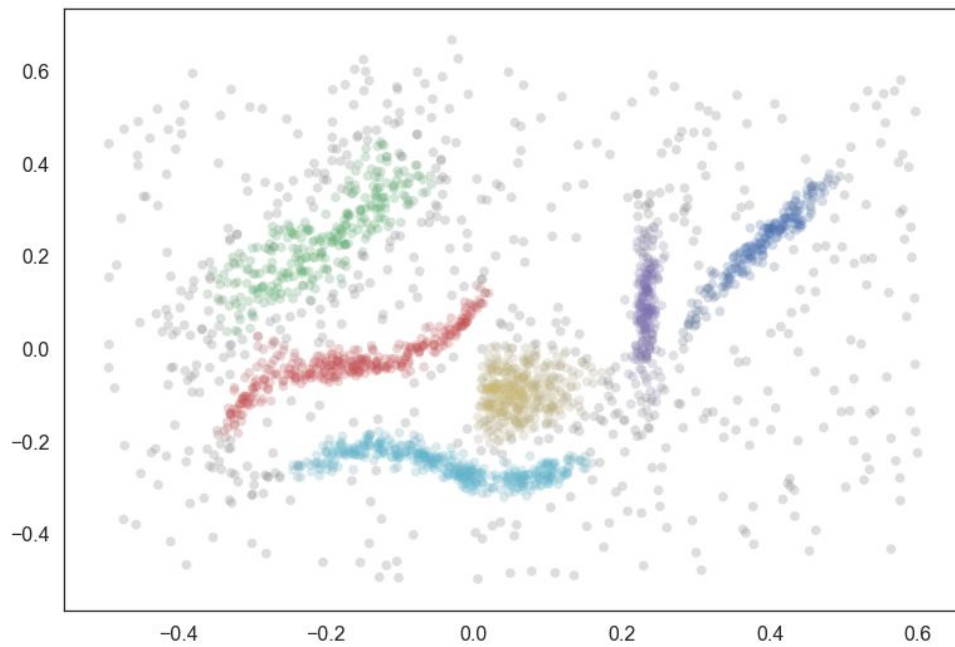
PORTLAND, Ore. -- First computers beat the best of us at chess, then poker, and finally Jeopardy. The next hurdle is image recognition — surely a computer can't do that as well as a human.

Tektronix
INFINITY & BEYOND
Go further, with 6 integrated instruments
CHOOSE YOUR SCOPE

RAW DATA VS FEATURES

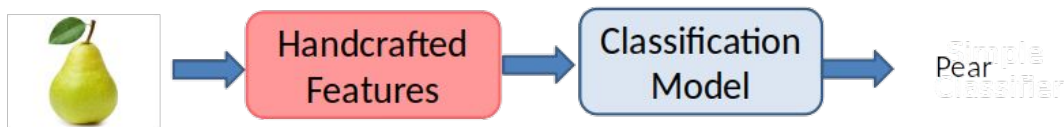


RAW DATA VS FEATURES

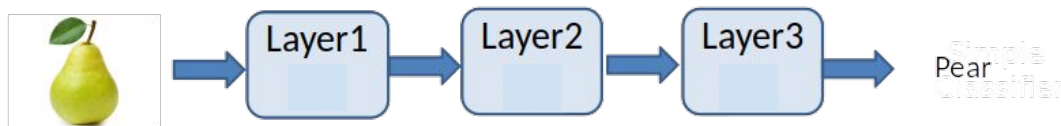


MACHINE LEARNING VS DEEP LEARNING

- *Traditional Machine Learning*

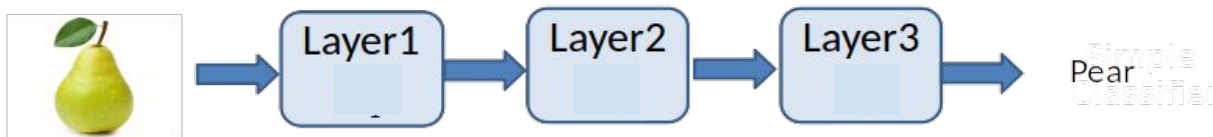


- *Deep Learning*

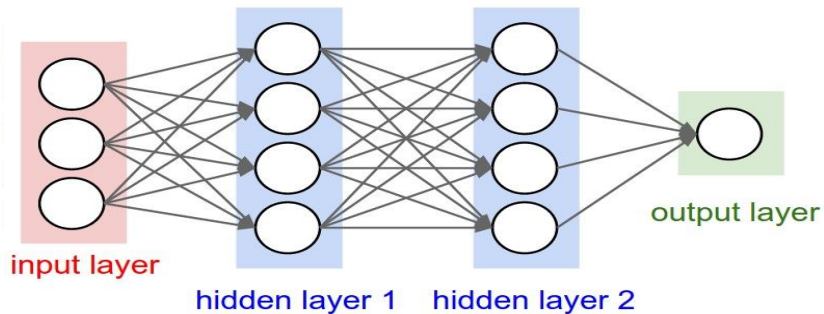


MODULAR LEARNING

- A neural network is:
 - a composition of modules (building blocks)
 - a series of hierarchically connected functions, each one with its own parameters



$$f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$$

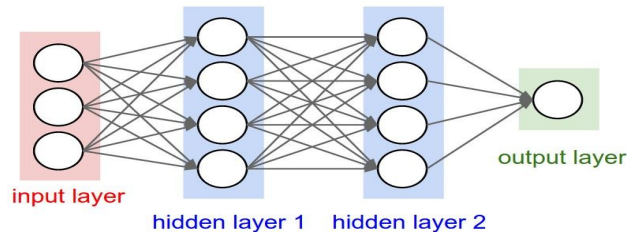


FEEDFORWARD NETWORKS: BASIC ELEMENTS

FEEDFORWARD NETWORKS

- Goal: Approximate some unknown ideal function $f^* : \mathcal{X} \rightarrow \mathcal{Y}$
- Feedforward Network:
 - Define parametric mapping $f(\mathbf{x}_i; \Theta)$
 - Learn parameters to get a good approximation of f^* from available sample
- Information flow in function evaluation begins at input, flows through intermediate computations to produce the output
- The function f is a composition of multiple functions:

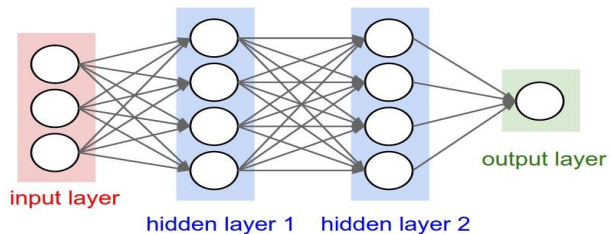
$$f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$$



FEEDFORWARD NETWORKS

- Function composition can be described by a Directed Acyclic Graph
- $f^{(1)}$ is the first layer, $f^{(2)}$ the second layer and so on.
- Depth is the maximum i in the function composition chain
- Final layer is called the output layer

$$f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$$



TRAINING

- Training: Optimize Θ to drive $f(\mathbf{x}; \Theta)$ closer to $f^*(\mathbf{x})$
- Training Data: $f^*(\mathbf{x})$ evaluated at different $\mathbf{x}$ instances (i.e. expected outputs)
- Only specifies the output of the output layers
- Output of intermediate layers is not specified, hence the nomenclature hidden layers

TRAINING

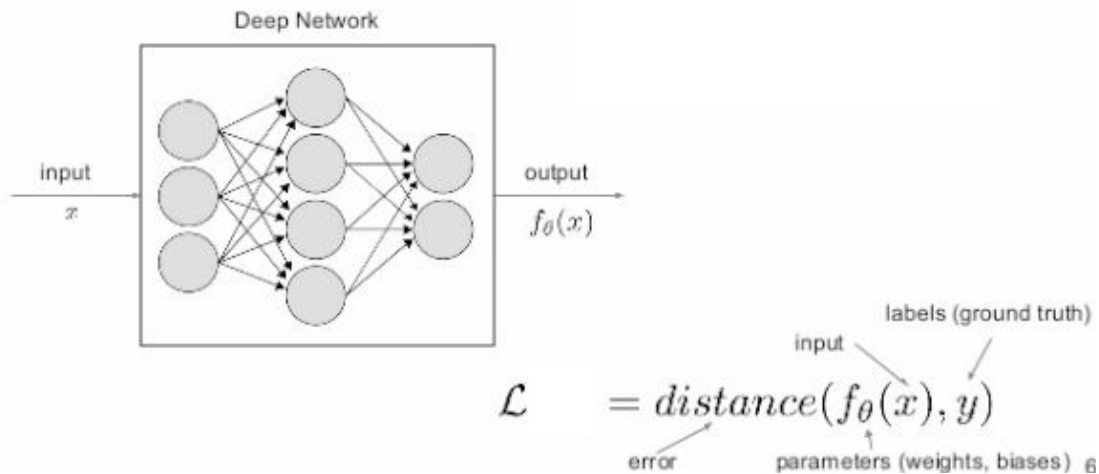
- Designing and training a neural network is not much different from training any other machine learning model with gradient descent
 - The problem becomes non-convex
 - No convergence guarantees
- To apply gradient descent: need to specify a model, a cost function and the output representation

MODELING CHOICES

- Need to choose:
 - Cost function
 - Form of output
 - Activation functions
 - Architecture (number of layers etc)
 - Optimizer

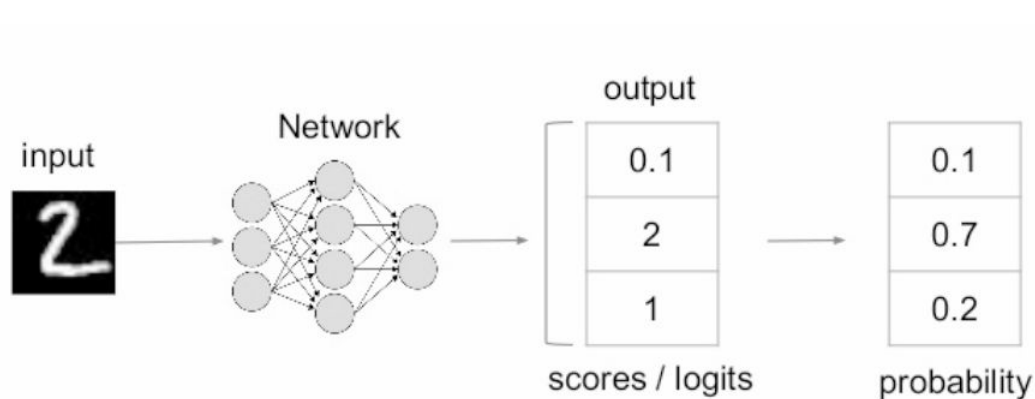
COST FUNCTION

- How good does the network on training data?



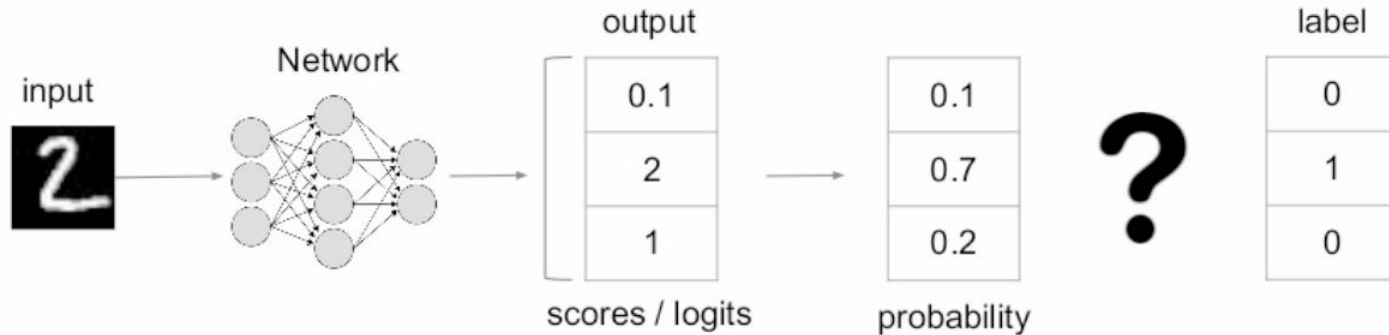
COST FUNCTION

- We can apply the losses we have seen so far, e.g. square loss for regression
- For classification is common to convert outputs into probabilities: **softmax**



$$S(l_i) = \frac{e^{l_i}}{\sum_k e^{l_k}}$$

COST FUNCTION: CROSS-ENTROPY



$$\mathcal{L}_i = - \sum_k y_k \log(S(l_k)) = - \log(S(l))$$

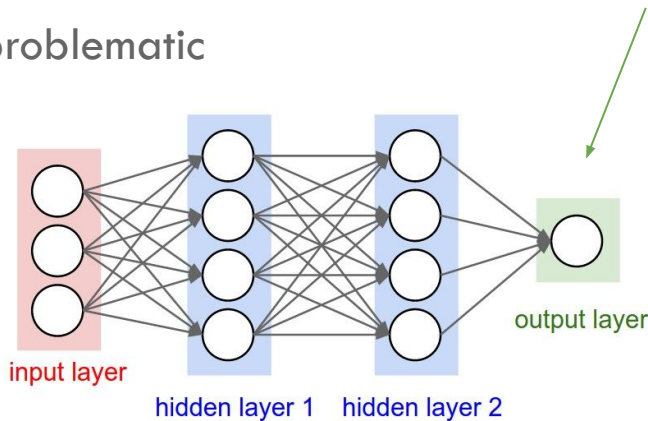
- The choice of the loss is related to the choice of the output unit.

OUTPUT UNITS: LINEAR

- Given features h , a layer of linear output units gives:

$$\hat{y} = W^T h + b$$

- Linear units do not saturate, little difficulty for gradient-based optimization algorithms
- A gradient of the output of the model close to 0 is problematic



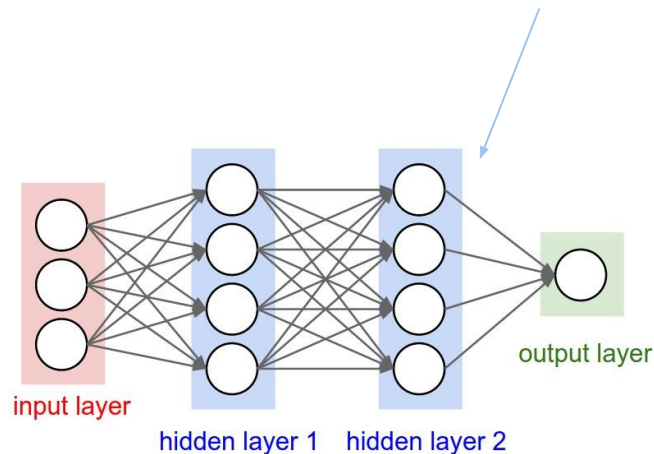
OUTPUT UNITS: SOFTMAX

- It is sometimes desirable to produce normalized probabilities in the output layer.
- Linear layer produces unnormalized log probabilities
- Softmax (as shown before):

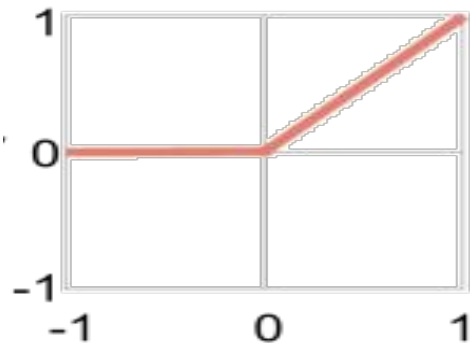
$$\overset{\text{scores (logits)}}{S(l_i)} = \frac{e^{l_i}}{\sum_k e^{l_k}}$$

HIDDEN UNITS

- Inside an hidden unit...
 - Accept input $\mathbf{x}$
 - Compute affine transformation $\mathbf{z} = W^T \mathbf{x} + \mathbf{b}$
 - Apply elementwise non-linear function $h(z)$
 - Obtain output $h(z)$
- Choices for h ?
- Design of hidden units is an active area of research



RECTIFIED LINEAR UNITS (RELU)



$$h(x) = \max(0, x)$$

- Gradient is 0 or 1
- Similar to linear units. Easy to optimize!
- Give large and consistent gradients when active
- Not everywhere differentiable. In practice not a problem: return one sided derivatives at $z = 0$. Gradient based optimization is subject to numerical error anyway.

GENERALIZED RELU

- Several variations to ReLU
- Solve the problem that units die when the gradient is 0.
- Some examples:

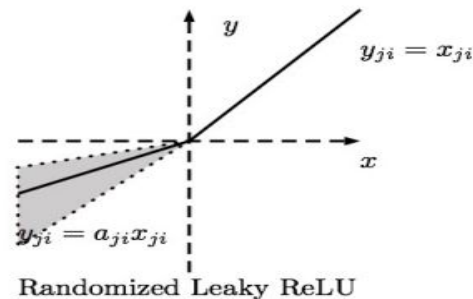
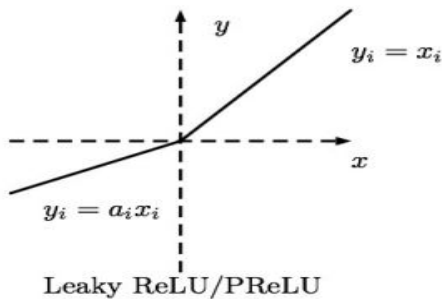
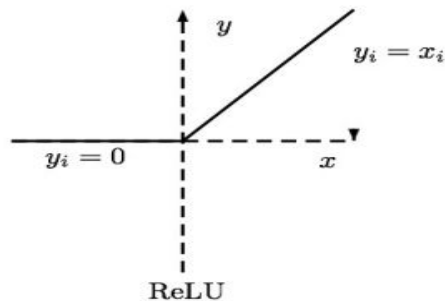
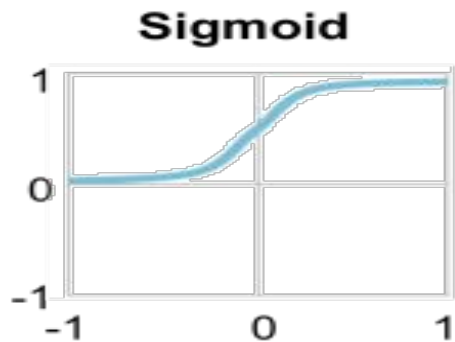


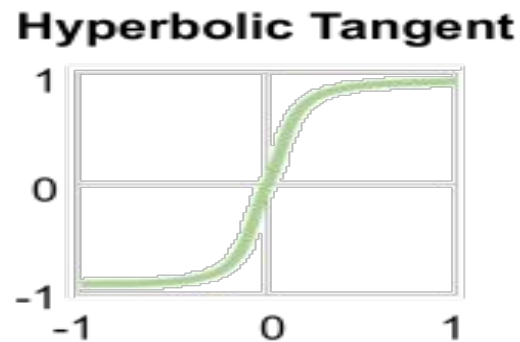
Figure: Xu *et al.* "Empirical Evaluation of Rectified Activations in Convolutional Network"

SIGMOID AND TANH

- Squashing type of non-linearity: pushes outputs to range $[0,1]$ or $[-1,1]$
- Problem: Saturate across most of the domain, strongly sensitive only when the input is closer to zero
- Saturation makes gradient based learning difficult



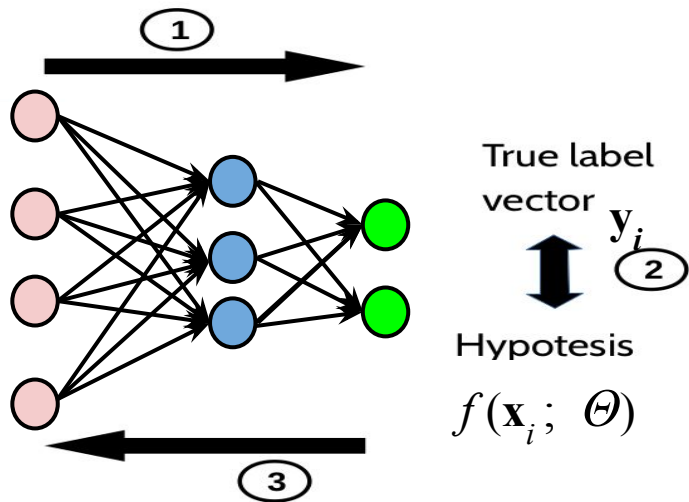
$$h(x) = \frac{1}{1 + e^{-x}}$$



$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ARCHITECTURE

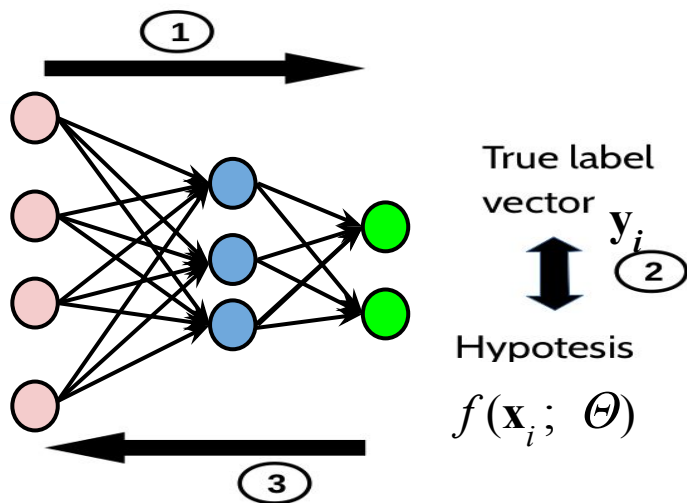
- How do we decide depth and width of a neural network?
- Few theoretical results, mostly based on empirical findings
- **Theoretical result** [Cybenko, 1989]: *2-layer net with linear output with some squashing non-linearity in hidden units can approximate any continuous function over compact domain to arbitrary accuracy (given enough hidden units!)*
- Theorem also holds for other non-linear functions (e.g. ReLU)
- Implication: regardless of function we are trying to learn, we know a large MLP can represent this function
- However, we are not guaranteed that our training algorithm will be able to learn that function (e.g. problems in optimization, overfitting)



BACKPROPAGATION

LEARNING WEIGHTS

How do we learn the weights?



- Step 2: Compare outputs with correct answer to get **error signal**
- Step 3: **Backpropagate** error signal to get derivatives for learning

BACKPROPAGATION: THREE STEPS

- Step 1: **Feedforward Propagation**: Accept input $\mathbf{x}$, pass through intermediate stages and obtain output
- Step 2: Use the computed output to **compute a scalar cost** depending on the loss function
- Step 3: **Backpropagation** allows information to flow backwards from cost to compute the gradient

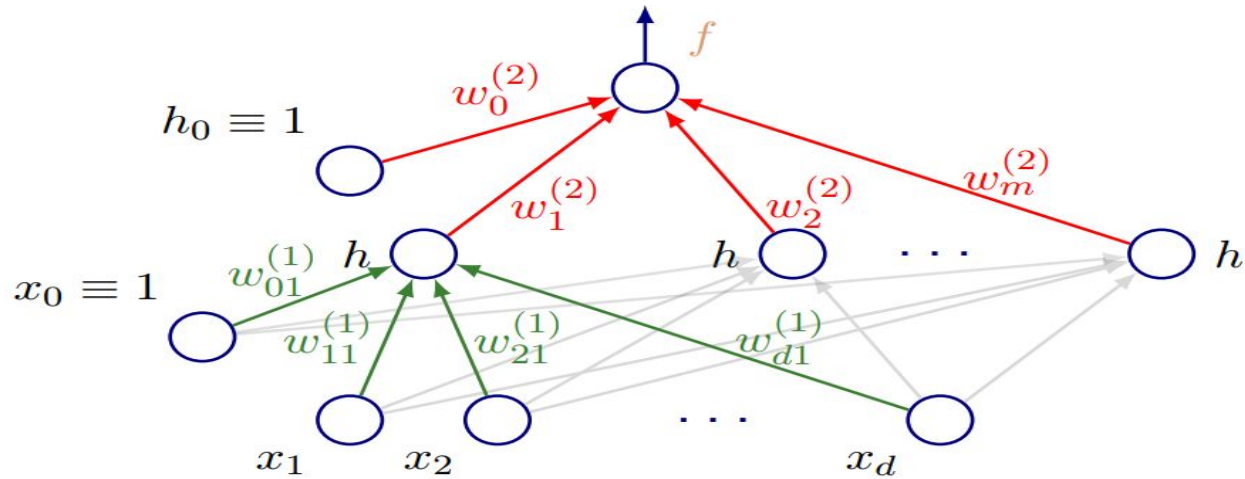
BACKPROPAGATION: KEY IDEAS

- We use **gradient descent**: we need error derivatives for all the weights in the net.
- Example (first layer):

$$w_{11}^{(1)} := w_{11}^{(1)} - \eta \frac{\partial L}{\partial w_{11}^{(1)}}$$

- From the training data we do not know what the **hidden units should do** but, we can compute how fast the error changes as we change a hidden activity
- **Use error derivatives w.r.t. hidden activities**
- Each hidden unit can affect many output units and have **separate effects on error**. We combine these effects.
- We can **compute error derivatives for hidden units efficiently**. Once we have error derivatives for hidden activities, easy to get error derivatives for weights going in.

STEP 1: FEEDFORWARD OPERATION



From input to output:

$$\hat{y}(\mathbf{x}; \mathbf{w}) = f \left(\sum_{j=1}^m w_j^{(2)} h \left(\sum_{i=1}^d w_{ij}^{(1)} x_i + w_{0j}^{(1)} \right) + w_0^{(2)} \right)$$

STEP 2: COMPUTE ERROR AND TRAIN

Error of the network on a training set:

$$L(X; \mathbf{w}) = \sum_{i=1}^N \frac{1}{2} (y_i - \hat{y}(\mathbf{x}_i; \mathbf{w}))^2$$

No closed-form solution. To train resort to gradient descent.

Need to evaluate derivative of L on a single example.

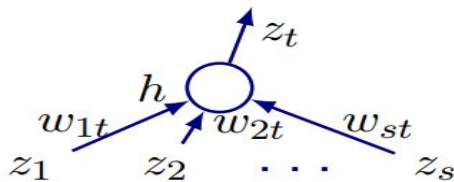
We can consider a simple linear model for output $\hat{y} = \sum_j w_j x_{ij}$:

$$\frac{\partial L(\mathbf{x}_i)}{\partial w_j} = \underbrace{(\hat{y}_i - y_i)}_{\text{error}} x_{ij}.$$

STEP 3: BACKPROPAGATION

General unit activation in a multilayer network:

$$z_t = h \left(\sum_j w_{jt} z_j \right)$$



Forward propagation: calculate for each unit

$$a_t = \sum_j w_{jt} z_j$$

The loss L depends on w_{jt} only through a_t

$$\frac{\partial L}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} \frac{\partial a_t}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} z_j$$

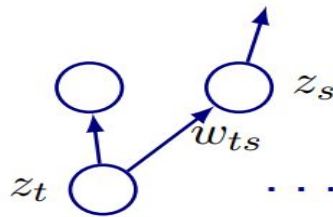
STEP 3: BACKPROPAGATION

$$\frac{\partial L}{\partial w_{jt}} = \underbrace{\frac{\partial L}{\partial a_t}}_{\delta_t} z_j$$

Output unit with linear activation $\delta_t = \hat{y} - y$

Hidden unit t which sends output to units S :

$$\begin{aligned}\delta_t &= \sum_{s \in S} \frac{\partial L}{\partial a_s} \frac{\partial a_s}{\partial a_t} \\ &= h'(a_t) \sum_{s \in S} w_{ts} \delta_s\end{aligned}$$



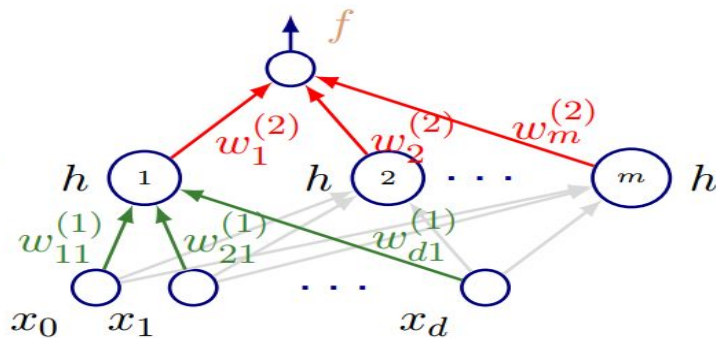
$$a_s = \sum_{j: j \rightarrow s} w_{js} h(a_j)$$

EXAMPLE

- Output: $f(a) = a$
- Hidden:

$$h(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}},$$

$$h'(a) = 1 - h(a)^2.$$



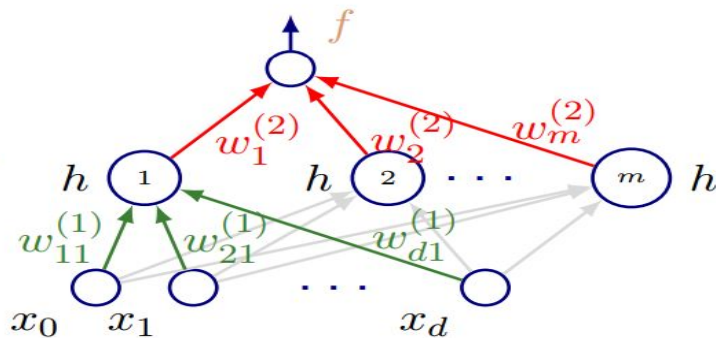
EXAMPLE

- Output: $f(a) = a$

- Hidden:

$$h(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}},$$

$$h'(a) = 1 - h(a)^2.$$



- Given $\mathbf{x}$, feedforward inputs

Input to hidden

$$a_j = \sum_{i=0}^d w_{ij}^{(1)} x_i,$$

Hidden output

$$z_j = \tanh(a_j),$$

Net output

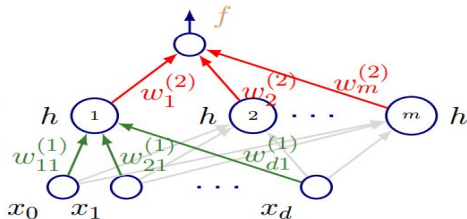
$$\hat{y} = a = \sum_{j=0}^m w_j^{(2)} z_j.$$

EXAMPLE

$$a_j = \sum_{i=0}^d w_{ij}^{(1)} x_i, \quad z_j = \tanh(a_j), \quad \hat{y} = a = \sum_{j=0}^m w_j^{(2)} z_j.$$

Error on example $\mathbf{x}$: $L = \frac{1}{2}(y - \hat{y})^2$.

Output unit: $\delta = \frac{\partial L}{\partial a} = y - \hat{y}$.



EXAMPLE

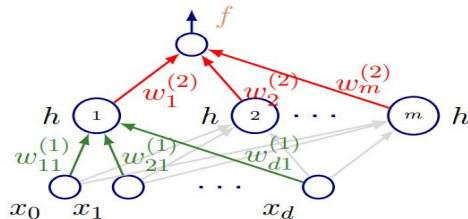
$$a_j = \sum_{i=0}^d w_{ij}^{(1)} x_i, \quad z_j = \tanh(a_j), \quad \hat{y} = a = \sum_{j=0}^m w_j^{(2)} z_j.$$

Error on example $\mathbf{x}$: $L = \frac{1}{2}(y - \hat{y})^2$.

Output unit: $\delta = \frac{\partial L}{\partial a} = y - \hat{y}$.

Next, compute δ for the hidden units:

$$\delta_j = (1 - z_j)^2 w_j^{(2)} \delta$$



Derivative with respect to weights:

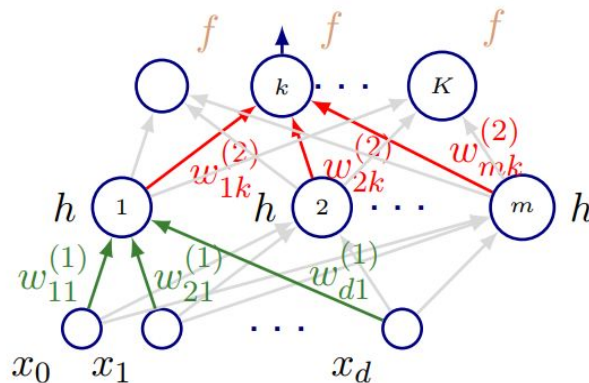
$$\frac{\partial L}{\partial w_{ij}^{(1)}} = \delta_j x_i, \quad \frac{\partial L}{\partial w_j^{(2)}} = \delta z_j.$$

Update weights: $w_j \leftarrow w_j - \eta \delta z_j$ $w_{ij}^{(1)} \leftarrow w_{ij}^{(1)} - \eta \delta_j x_i$.

EXAMPLE: MULTIDIMENSIONAL OUTPUT

Loss on example

$$\frac{1}{2} \sum_{k=1}^K (y_k - \hat{y}_k)^2$$

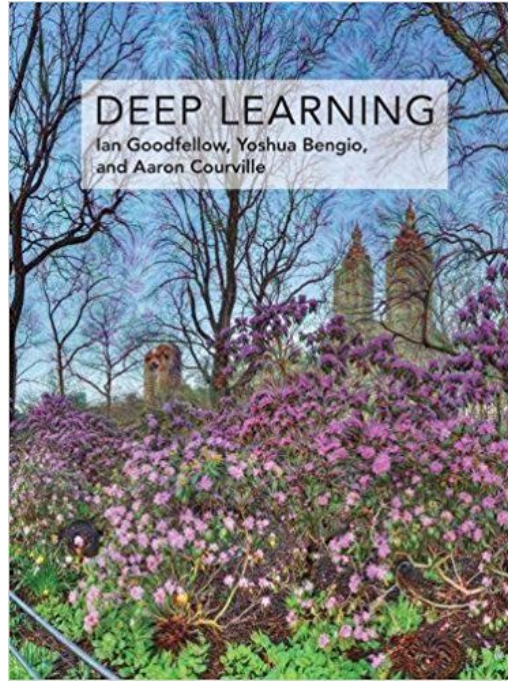


Now, for each output unit $\delta_k = y_k - \hat{y}_k$;

For hidden unit j :

$$\delta_j = (1 - z_j)^2 \sum_{k=1}^K w_{jk}^{(2)} \delta_k.$$

BOOK



<https://www.deeplearningbook.org/contents/mlp.html>

QUESTIONS?

